

Git-Lite — Mini Version Control System

What is Git-Lite?

Git-Lite is a small Python project inspired by Git. The main idea is to understand **how version control works internally** instead of just using Git commands.

It allows us to save versions of files, keep track of previous commits, create branches, switch between them, and go back to an older version of the project.

For example:

```
Edit file
  ↓
Create Commit
  ↓
Save file version
  ↓
Create unique ID
  ↓
Later change the file
  ↓
Rollback
  ↓
Get the old file back
```

DSA Concepts Used

1. Hashing

What is it?

Hashing converts data into a unique-looking value called a **hash**.

For example:

```
"Hello Git-Lite!"
  ↓
63702d2369
```

Instead of using the complete file content every time, we can use its hash to identify that content.

How we used it?

We use hashing for:

Yateesh (92400118666)

- Creating commit IDs
- Creating IDs for file contents
- Finding stored file versions

Our project uses SHA-256 hashing.

```
hashlib.sha256(data.encode()).hexdigest()
```

So hashing is one of the main concepts behind our project.

2. Hash Map / Dictionary

What is it?

A hash map stores data in **key-value pairs**.

Example:

```
{ "notes.txt": "08b8546d4b" }
```

Here:

Key → notes.txt

Value → file's object ID

How we used it?

We use Python dictionaries to store:

File → Object ID

notes.txt → 08b8546d4b

We also use a dictionary for branches:

main → commit ID

feature → commit ID

This makes it easy to find the commit associated with a file or branch.

3. Linked List Concept

What is it?

In a linked list, one element stores a reference to another element.

For example:

Node A → Node B → Node C

How we used it?

Our commits work in a similar way.

Every commit stores its **parent commit ID**.

Example:

```
Commit C
  ↓ parent
Commit B
  ↓ parent
Commit A
```

So when we want to see the history, we start from the current commit and keep following the parent.

```
current = commit["parent"]
```

This is similar to traversing a linked list.

4. Graph

What is it?

A graph consists of **nodes and connections between them**.

In our project:

```
Commit A
  ↓
Commit B
  ↓
Commit C
```

Each commit is a node and the parent relationship is an edge.

When branches are created, the structure becomes more like a graph:

```
          → Commit C → Commit D
        /
Commit A → Commit B
```

The branches can point to different commits in this structure.

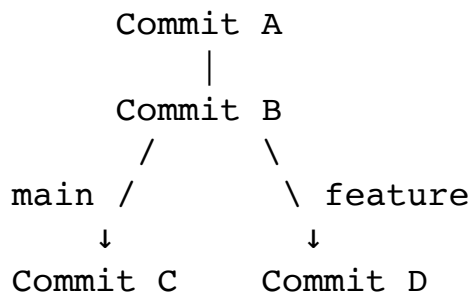
How we used it?

Our commit history forms a **version-history graph**, and we traverse it through parent commit IDs.

5. Tree-like Structure

We don't directly implement a formal tree data structure, but our project has a **tree-like version structure** when branches split.

For example:



This helps demonstrate how different development paths can come from the same earlier commit.

6. Traversal

What is it?

Traversal means **visiting elements one by one** to reach or process the required data.

How we used it?

When the user selects **View Commit History**, we start at the current commit and repeatedly follow its parent:

Current Commit

↓

Parent

↓

Parent

↓

Parent

↓

None

Our code does this with:

```
while current:    commit = load_commit(current)    current = commit["parent"]
```

So this is a simple example of **linked-structure traversal**.

7. File System / Storage

This isn't strictly a DSA, but it's an important part of the project.

We separate the actual file contents from the commit information.

For example:

Commit

↓

notes.txt → 08b8546d4b

↓

data/objects/

↓

VERSION 2 – NEW CONTENT

This allows us to restore an older version when rollback is performed.

Simple Summary for Your DSA File

Git-Lite is a Python-based mini version control system developed to understand how Git works internally. It allows users to create commits, track files, maintain commit history, create and switch branches, and rollback to previous versions. The project uses important DSA concepts such as hashing, hash maps, linked structures, graphs, and traversal. Hashing is used to generate unique IDs for commits and file contents, dictionaries are used to store branches and file mappings, and each commit stores its parent commit ID, creating a linked history. The commit history can then be traversed by following these parent links.

DSA concepts in one table

DSA	How we used it
Hashing	Generate IDs for commits and file contents
Hash Map	Store branches and file → object ID mappings
Linked List Concept	Each commit points to its parent
Graph	Commits and branches form a version-history structure
Traversal	Follow parent commits to display history
Tree-like Structure	Branches create different paths from commits